

Programování - 3. ročník

První hodina: diagnostická úloha Stav projektu

Cílem není závod ani známka za rychlost. Postupuj po úrovních a skonči tam, kde už řešení nedokážeš samostatně dokončit a vysvětlit. Právě to ukáže, na čem budeme dále pracovat.

Co v materiálu najdeš

- kontrolu pracovního prostředí a klasifikační podmínky;
- zadání rozdělené do pěti navazujících úrovní;
- teorii vstupů, datových typů, podmínek, validace, výjimek, cyklů a funkcí;
- komentované základní i kompletní vzorové řešení;
- kontrolní otázky, testovací případy a sebehodnocení.

Kontrola programového vybavení

- Python 3 je nainstalovaný a příkaz `python --version` nebo `python3 --version` funguje.
- Dokážeš vytvořit, uložit a spustit soubor s příponou `.py`.
- Máš funkční vývojové prostředí, například Visual Studio Code nebo PyCharm.
- Funguje terminál a Git; máš přístup k výukovým materiálům a místu pro odevzdání.

```
import sys

# Vypišeme verzi Pythonu, kterou program skutečně používá.
print("Verze Pythonu:", sys.version)

# Pokud se zobrazí tento text, základní spuštění funguje.
print("Programovací prostředí funguje.")
```

Při problému vždy uveď operační systém, použitý příkaz, celé chybové hlášení a co už jsi vyzkoušel/a.

Klasifikační podmínky

Oblast	Podíl
Praktické programovací úlohy	40 %
Dlouhodobý nebo pololetní projekt	35 %
Samostatné kontrolní práce	20 %
Dokumentace, prezentace a průběžná aktivita	5 %

Odevzdává se zdrojový kód, nikoli pouze snímek obrazovky. Program musí být možné spustit podle návodu a autor musí rozumět podstatným částem svého řešení. Použité zdroje, knihovny a pomoc AI musí být přiznané.

Diagnostická úloha - Stav projektu

Vytvoř konzolový program, který načte název projektu, počet všech plánovaných úkolů a počet dokončených úkolů. Program vypočítá procento dokončení a slovně popíše stav projektu.

Pracuj samostatně. Každá dokončená úroveň musí být spustitelná a musíš dokázat vysvětlit, jak funguje.

Příklad běhu

```
Název projektu: Školní web
Počet všech úkolů: 10
Počet dokončených úkolů: 4

Projekt: Školní web
Dokončeno: 4 z 10 úkolů
Úspěšnost: 40 %
Hodnocení: projekt je na začátku
```

Úroveň 1 - vstup a výpočet

Načti tři vstupy, vypočítej procento dokončení a vypiš srozumitelný výsledek.

- Program lze spustit.
- Přijme název, počet všech úkolů a počet dokončených úkolů.
- Správně vypočítá procenta.
- Používá input(), proměnné, int(), aritmetiku a print().

Úroveň 2 - slovní stav

Pomocí if, elif a else doplň slovní hodnocení.

- 0 % - projekt nebyl zahájen.
- 1-49 % - projekt je na začátku.
- 50-99 % - projekt je rozpracovaný.
- 100 % - projekt je dokončený.
- Otestuj alespoň hodnoty 0, 25, 50 a 100 %.

Úroveň 3 - kontrola vstupu

Program nesmí při chybném vstupu spadnout.

- Odmítne záporné počty.
- Odmítne nulu jako celkový počet úkolů.
- Odmítne více dokončených úkolů, než je celkový počet.
- Zachytí slovo zadané místo celého čísla a vyžádá si hodnotu znovu.

Úroveň 4 - další projekt

Po vyhodnocení se program zeptá, zda má zpracovat další projekt.

- Při odpovědi ano se zadávání zopakuje.
- Jiná odpověď program ukončí.
- Použij cyklus while a řízené ukončení pomocí break.

Úroveň 5 - funkce

Přesuň opakované nebo logicky samostatné části do funkcí.

- Funkce má jeden jasný účel a vhodný název.
- Vstupy přijímá pomocí parametrů.
- Výsledek předává pomocí `return`.
- Hlavní část programu je díky funkcím kratší a čitelnější.

Teorie 1 - vstup, proměnné a datové typy

`input()` zobrazí výzvu, počká na uživatele a vždy vrátí text typu `str`. I když uživatel napíše číslice, program je nejprve obdrží jako text.

Proměnná je jméno odkazující na hodnotu. Název má popisovat význam: `celkem` je srozumitelnější než `x`. Python určuje typ hodnoty za běhu programu.

`int()` převádí vhodný text na celé číslo. Převod slova nebo desetinného zápisu vyvolá výjimku `ValueError`.

Výpočet procent

```
procenta = hotovo / celkem * 100
```

Operátor `/` provádí dělení. Podíl dokončených a všech úkolů násobíme stem. Hodnota `celkem` nesmí být nula, protože dělení nulou není definováno a v Pythonu vyvolá `ZeroDivisionError`. Funkce `round()` může výsledek zaokrouhlit.

Teorie 2 - podmínky a porovnání

Podmínka umožňuje zvolit další postup podle pravdivosti logického výrazu. Python vyhodnocuje větve shora dolů a provede první větev, jejíž podmínka je pravdivá. Větev `else` zachytí všechny zbývající případy.

```
if procenta == 0:
    print("Projekt nebyl zahájen.")
elif procenta < 50:
    print("Projekt je na začátku.")
else:
    print("Projekt je rozpracovaný nebo hotový.")
```

- `==` porovnává hodnoty; jedno `=` hodnotu přiřazuje.
- `< a >` znamenají menší a větší než.
- `<= a >=` zahrnují rovnost.
- Odsazení není pouze vzhled - v Pythonu určuje blok kódu a je součástí syntaxe.

Teorie 3 - validace a výjimky

Validace kontroluje, zda vstup splňuje pravidla úlohy. Údaj může mít správný datový typ, ale přesto být věcně chybný - například 15 dokončených úkolů z deseti.

Výjimka je objekt popisující mimořádnou situaci za běhu programu. Blok `try` obsahuje rizikovou operaci; `except ValueError` určuje reakci pouze na chybný převod čísla.

```
try:
    cislo = int(input("Zadej číslo: "))
except ValueError:
    print("Nebyl zadán platný celočíselný údaj.")
```

Dobrá praxe: zachytávej konkrétní výjimku, vypiš srozumitelnou zprávu a nech uživatele vstup opravit. Obecné `except` bez uvedení typu může skrýt skutečnou programátorskou chybu.

Teorie 4 - cyklus a funkce

Cyklus `while` opakuje blok, dokud je podmínka pravdivá. Konstrukce `while True` vytváří záměrně nekonečný cyklus; ukončení musí zajistit `break` nebo `return`.

Funkce je pojmenovaný blok kódu určený k opakování použití. Parametry jsou lokální jména pro vstupní hodnoty. `return` ukončí funkci a předá hodnotu volajícímu kódu. `break` naproti tomu ukončí pouze nejbližší cyklus.

```
def vypocitej_procenta(celkem, hotovo):
    # Funkce vrátí výsledek volajícímu kódu.
    return round(hotovo / celkem * 100)
```

Funkce má řešit jeden srozumitelný úkol. Menší funkce lze snadněji samostatně testovat, znovu použít a opravit.

Vzorový kód - základní řešení

Zobraz až po dokončení vlastního pokusu. Kód neopisuj bez porozumění; u každého řádku musíš znát jeho účel.

```
# Načteme název projektu jako text.
nazev = input("Název projektu: ")

# input() vrací text, proto číselné vstupy převedeme pomocí int().
celkem = int(input("Počet všech úkolů: "))
hotovo = int(input("Počet dokončených úkolů: "))

# Vypočítáme podíl dokončených úkolů a převedeme jej na procenta.
procenta = hotovo / celkem * 100

# Vypíšeme základní přehled.
print("Projekt:", nazev)
print("Dokončeno:", procenta, "%")

# Podmínka ověří, zda jsou dokončeny všechny úkoly.
if procenta == 100:
    print("Projekt je dokončený.")
else:
    print("Projekt ještě není dokončený.")
```

Co řešení zatím neumí

- Nezachytí slovo zadané místo čísla.
- Při celkovém počtu nula dojde k chybě dělení nulou.
- Přijme záporné nebo logicky nemožné hodnoty.
- Neumožňuje zadat více projektů bez nového spuštění.

Vzorový kód - kompletní řešení (1/2)

Všechny důležité kroky jsou v kódu komentované. Komentář vysvětluje důvod nebo princip; nemá pouze opakovat syntaxi řádku.

```
# Funkce načte nezáporné celé číslo.
# Parametr vyzva určuje text zobrazený uživateli.
def nacti_nezaporne_cislo(vyzva):
    # Cyklus pokračuje, dokud uživatel nezadá platnou hodnotu.
    while True:
        try:
            # input() vrátí text; int() jej převádí na celé číslo.
            cislo = int(input(vyzva))

            # Zápornou hodnotu nepřijmeme.
            if cislo < 0:
                print("Číslo nesmí být záporné.")
            else:
                # return ukončí funkci a vrátí platnou hodnotu.
                return cislo

        # ValueError vznikne například po zadání slova.
        except ValueError:
            print("Zadej celé číslo.")

# Funkce vypočítá procenta a určí slovní stav projektu.
def vypocitej_stav(celkem, hotovo):
    # Výsledek zaokrouhlíme na celé procento.
    procenta = round(hotovo / celkem * 100)

    # Podmínky se vyhodnocují shora dolů.
    if procenta == 0:
        stav = "projekt nebyl zahájen"
    elif procenta < 50:
        stav = "projekt je na začátku"
    elif procenta < 100:
        stav = "projekt je rozpracovaný"
    else:
        stav = "projekt je dokončený"

    # Funkce vrátí dvě hodnoty.
    return procenta, stav
```

Vzorový kód - kompletní řešení (2/2)

```
# Hlavní cyklus umožňuje vyhodnotit více projektů.
while True:
    print("\n--- Stav projektu ---")

    # strip() odstraní okrajové mezery.
    nazev = input("Název projektu: ").strip()

    # Prázdný název si vyžádáme znovu.
    while not nazev:
        print("Název nesmí být prázdný.")
        nazev = input("Název projektu: ").strip()

    celkem = nacti_nezaporne_cislo("Počet všech úkolů: ")

    # Nula by způsobila dělení nulou.
    while celkem == 0:
        print("Celkový počet musí být větší než nula.")
        celkem = nacti_nezaporne_cislo("Počet všech úkolů: ")

    hotovo = nacti_nezaporne_cislo("Počet dokončených úkolů: ")

    # Dokončených úkolů nemůže být více než všech.
    while hotovo > celkem:
        print("Dokončených úkolů nemůže být více než všech.")
        hotovo = nacti_nezaporne_cislo("Počet dokončených úkolů: ")

    # Převezmeme obě hodnoty vrácené funkcí.
    procenta, stav = vypocitej_stav(celkem, hotovo)

    # f-string vloží hodnoty proměnných přímo do textu.
    print(f"\nProjekt: {nazev}")
    print(f"Dokončeno: {hotovo} z {celkem} úkolů")
    print(f"Úspěšnost: {procenta} %")
    print(f"Hodnocení: {stav}")

    # lower() převede odpověď na malá písmena.
    pokracovat = input("\nZadat další projekt? ano/ne: ").strip().lower()

    # Jiná odpověď než ano ukončí hlavní cyklus.
    if pokracovat != "ano":
        print("Program byl ukončen.")
        break
```

Testování a dobrá praxe

Testovací případ je konkrétní vstup a očekávaný výsledek. Neověřuj pouze běžnou situaci; zkoušej také krajní a chybné hodnoty.

Vstup	Co ověřuje	Očekávané chování
10 / 0	dolní hranici	0 %, projekt nebyl zahájen
10 / 5	hranici 50 %	50 %, projekt je rozpracovaný
10 / 10	horní hranici	100 %, projekt je dokončený
0 / 0	dělení nulou	odmítnout celkový počet 0
10 / 15	logickou platnost	odmítnout hotovo > celkem
ahoj	chybný datový typ	zobrazit chybu a opakovat vstup

Kontrolní otázky

- Proč `input()` vrací `str`, i když uživatel zadá číslice?
- Jaký je rozdíl mezi `=` a `==`?
- Proč musí být případy v řetězci `if/elif` seřazeny promyšleně?
- Jaký je rozdíl mezi syntakticky platným a věcně platným vstupem?
- Kdy vznikne `ValueError` a kdy `ZeroDivisionError`?
- Jak se liší `break` a `return`?
- Proč jsou funkce výhodné pro testování a údržbu programu?
- Kde v kompletním řešení funkce vrací dvě hodnoty?

Odevzdání a sebehodnocení

Odevzdej soubor `main.py`. Před odevzdáním program alespoň třikrát spust' s různými vstupy. Na konec souboru přidej komentář:

```
# Poslední plně dokončená úroveň:  
# Částečně dokončená úroveň:  
# Na čem jsem se zastavil/a:  
# Jaká chyba se objevila:  
# Co jsem vyzkoušel/a:  
# Co bych dokázal/a vysvětlit:  
# S čím potřebuji pomoci:
```

Kód, který nedokážeš vysvětlit, se nepovažuje za zvládnutou úroveň.

Mgr. Libor Čamek